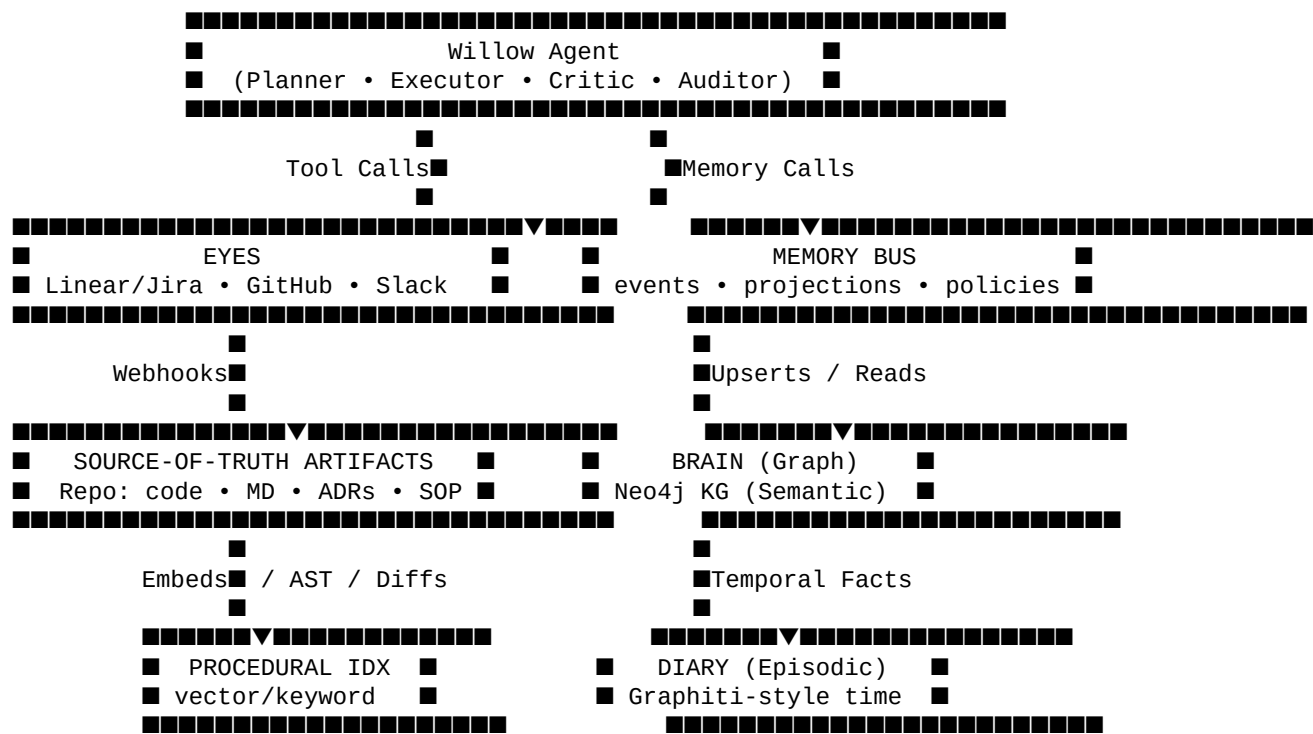


Willow: Autonomous Memory Stack — Architecture Focus (2025–2026)

Purpose: visualize a practical, best-in-class memory + state-sync architecture for Willow (Brain/Diary/Eyes) and resolve the mismatch between Semantic Memory (graph) and Procedural Memory (markdown/code).

1) Willow System Map



2) Core Architectural Principles

Principle	What it means
Single Writer, Many Readers	Avoid letting multiple components directly mutate the same truth. Treat external artifacts (repo, tickets)
Event-Sourced Sync	Use a Memory Bus with append-only events (webhooks, git commits, doc edits). Build projections: (a) K
Bidirectional, but Asymmetric	KG can suggest edits to docs, but commits still go through standard workflows (PRs). External artifacts
Temporal + Typed Memory	Store 'facts' with validity windows (from/to), and store 'episodes' as immutable entries with links to facts
Diff-First Updates	Prefer diffs/patches over re-ingesting entire files. Map code/MD to stable IDs (paths + symbol Ds + sec

3) State Sync Pattern That Works in Practice

Your pain point is KG (semantic) drifting from docs/code (procedural). The fix is to formalize synchronization as: **Event** → **Normalize** → **Decide** → **Project**.

- Event Sources
- Git push / PR merged
 - Markdown edit (repo commit)
 - Linear/Jira ticket change
 - CI output (tests, deploys)

■
▼
[Normalizer]
- compute diff (git)
- parse symbols (AST) & doc anchors
- extract entities (LLM/NER) + metadata

■
▼
[Decision Layer: "Policy Engine"]
- what becomes a FACT? (semantic)
- what becomes an EPISODE? (timeline)
- what becomes an INDEX item? (vector)
- conflict rules (canonical wins)

■
▼
Projections (Derived States)
A) Brain (Neo4j KG): entities/relations + validity windows
B) Diary (Episodic): episode nodes w/ links to affected facts
C) Procedural Index: embeddings + keyword index + symbol index
D) Observability: traces, diffs, provenance graphs

4) Concrete Mechanics for Markdown/Code ■ Graph Consistency

- A) Stable IDs
- Code: (repo, path, symbol_id) where symbol_id = language server / AST hash
 - Markdown: (repo, path, heading_anchor) where anchor is generated + committed
- B) Provenance Edges
- Every fact edge/node stores: source_type (code/md/ticket), source_id, commit_sha, line_range/anchor, t
 - This enables "explain why Willow thinks X" and easy invalidation when source changes.
- C) Diff Ingestion
- On commit: compute changed files + unified diff.
 - For each changed region: update only the impacted symbol_ids / anchors.
 - Mark prior graph assertions from the old commit_sha as expired (valid_to = commit_time).
- D) Reconciliation Rules (asymmetric truth)
- Canonical sources: repo + tickets.
 - Graph is a projection: if graph conflicts, it is repaired by re-ingesting from canonical, not manual e
- E) Suggest, Don't Mutate
- If Willow wants to update docs, it opens a PR with a patch.
 - Once merged, the ingestion pipeline updates the graph.

5) Willow-Specific: Brain • Diary • Eyes as a Unified Memory System

Willow Components (your metaphor)

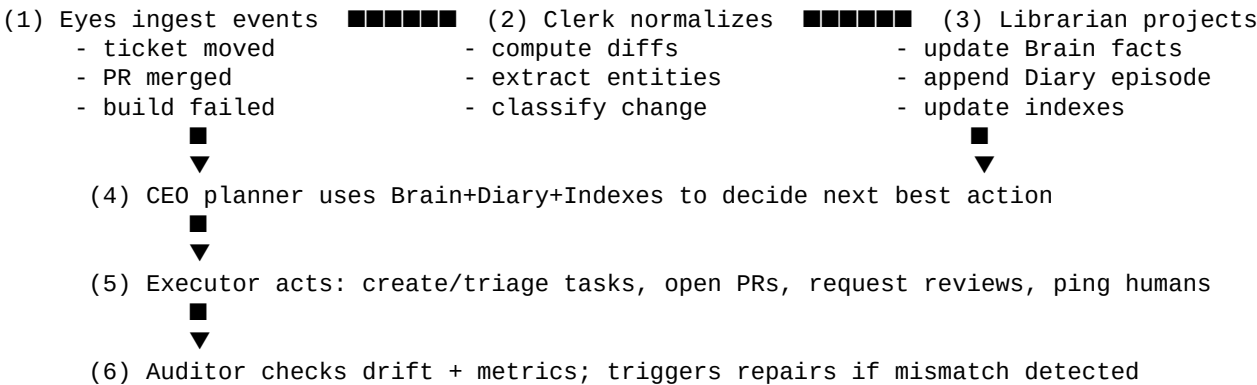
- Brain = Semantic Memory (Neo4j Knowledge Graph)
- Diary = Episodic Memory (Graphiti-style timeline + causality)
- Eyes = External state sensors (Linear/Jira + Git + CI + Docs)

Missing piece you need:
"Memory Bus" = the sync spine between Eyes ↔ Artifacts ↔ Brain/Diary

Willow runs best as a **multi-loop agent** with explicit roles:

Role	Responsibility
Perceiver	Consumes events from Eyes (webhooks, commits, ticket updates).
Clerk	Normalizes events into structured deltas (diffs, extracted entities, changed sections).
Librarian	Updates Brain/Diary projections according to policies; manages invalidation.
CEO	Plans, prioritizes, and delegates work using the current state.
Auditor	Continuously checks for drift: graph vs artifacts; opens repair PRs or tickets.

6) Willow Runtime Loop (What Happens Every Minute)



7) Drift Detection (The Practical Fix for Your Current Issue)

Add an explicit "Drift Scanner" job:

Inputs:

- latest repo HEAD (markdown + code)
- current graph snapshot (or incremental)

Checks:

- Are there graph facts with source_id that no longer exists?
- Do doc anchors/symbol_ids still match?
- Do tickets referenced in graph still exist / changed status?

Actions:

- expire invalid facts
- re-ingest impacted artifacts
- open a "Sync Repair" PR or Jira/Linear ticket

8) Minimal Implementation Blueprint (Python)

Services

- webhook-service: receives Linear/Jira/GitHub events → publishes to queue (Redis/Kafka)
- normalizer-worker: consumes events → emits structured deltas (diffs, entities, anchors)
- projector-worker: writes to Neo4j + episodic store + vector index

- drift-scanner (cron): validates invariants, triggers repair

Data Stores

- Neo4j: Brain (entities, relations, validity windows, provenance)
- Postgres: Diary episodes + event log (append-only)
- Vector DB (pgvector/Qdrant): procedural text + summaries
- Object store: raw artifacts snapshots (optional)

Key invariants

- every fact has provenance (source_type, source_id, commit_sha, timestamp)
- canonical artifacts win conflicts
- graph writes happen only via projector-worker (single-writer)